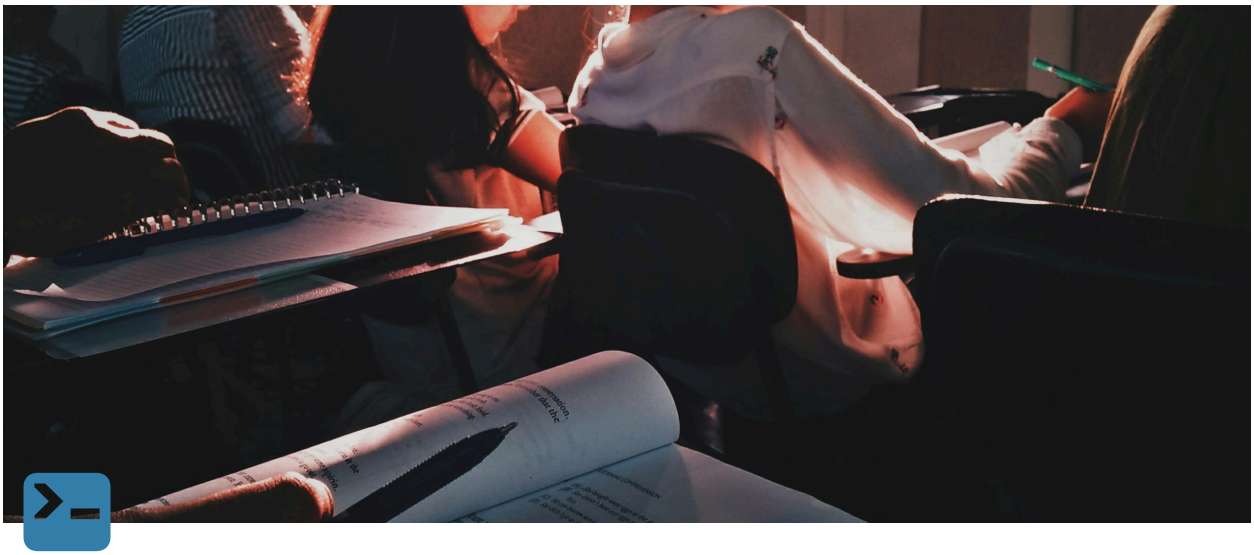

📄 PRUEBA MODULO 3

🏢 Caso de Estudio Propuesto — UrbanRent Analytics

> **Cobertura de sesiones:**
>
> - Clase 15 Sesión 2 — Spark DataFrames I: Introducción (crear, cargar, explorar)
> - Clase 16 Sesión 2 — Spark DataFrames I: Introducción (mismos contenidos, verificado)
> - Clase 17 Sesión 1 — Operaciones básicas (select, filter, withColumn, orderBy, drop, dropDuplicates)
> - Clase 17 Sesión 2 — Agregaciones (groupBy, agg, Window, rollup, cube, pivot)
> - Clase 18 Sesión 1 — Joins (inner, left, right, full, semi, anti, cross, broadcast)
> - Clase 19 Sesión 1 — Funciones integradas y UDFs (texto, fecha, arrays, UDF)

🌆 Contexto del caso

UrbanRent Analytics es una empresa española de alquiler de apartamentos turísticos con presencia en cinco ciudades. Gestiona tres tipos de información:

- Un fichero **CSV** con el registro de todas las **reservas** realizadas durante 2025.
- Un fichero **CSV** con el catálogo de **apartamentos** disponibles y sus características.
- Un fichero **JSON** con el perfil de los **propietarios** de cada apartamento.

El equipo de datos acaba de centralizar toda la información en Apache Spark y necesita construir un pipeline de análisis completo para responder preguntas estratégicas de la dirección.

> ⚠️ **Importante:** Las preguntas están agrupadas por sesión. Cada sección solo utiliza las herramientas vistas en esa sesión concreta. Trabaja en orden.

📦 Datos del caso

Crea los siguientes ficheros antes de abrir el notebook.

Fichero 1 — `reservas.csv`

```
id_reserva,id_apartamento,fecha_entrada,fecha_salida,num_huespedes,precio_noche,canal,val
oracion,ciudad
R001,APT001,2025-01-10,2025-01-13,2,95.0,Airbnb,4.8,Madrid
R002,APT002,2025-01-12,2025-01-15,4,120.0,Booking,4.5,Barcelona
R003,APT003,2025-01-20,2025-01-22,1,75.0,Directo,5.0,Valencia
R004,APT001,2025-02-01,2025-02-05,3,95.0,Airbnb,4.7,Madrid
R005,APT004,2025-02-10,2025-02-12,2,110.0,Booking,4.2,Sevilla
R006,APT002,2025-02-14,2025-02-17,5,120.0,Airbnb,4.9,Barcelona
R007,APT005,2025-02-20,2025-02-23,2,85.0,Directo,4.6,Bilbao
R008,APT003,2025-03-01,2025-03-04,1,75.0,Booking,4.4,Valencia
R009,APT001,2025-03-10,2025-03-14,4,95.0,Airbnb,4.8,Madrid
R010,APT006,2025-03-15,2025-03-17,2,130.0,Directo,5.0,Madrid
R011,APT004,2025-03-20,2025-03-22,3,110.0,Airbnb,4.3,Sevilla
R012,APT007,2025-04-01,2025-04-04,2,90.0,Booking,4.5,Barcelona
R013,APT005,2025-04-08,2025-04-10,1,85.0,Directo,4.7,Bilbao
R014,APT006,2025-04-12,2025-04-16,3,130.0,Airbnb,4.9,Madrid
R015,APT002,2025-04-20,2025-04-23,4,120.0,Booking,4.6,Barcelona
R016,APT008,2025-05-01,2025-05-04,2,70.0,Directo,4.1,Valencia
R017,APT003,2025-05-10,2025-05-13,2,75.0,Airbnb,4.5,Valencia
```

```
R018,APT007,2025-05-15,2025-05-18,3,90.0,Booking,4.8,Barcelona
R019,APT001,2025-05-20,2025-05-24,2,95.0,Directo,5.0,Madrid
R020,APT009,2025-05-25,2025-05-28,1,65.0,Airbnb,3.9,Bilbao
R021,APT006,2025-06-01,2025-06-05,4,130.0,Booking,4.7,Madrid
R022,APT010,2025-06-08,2025-06-10,2,80.0,Directo,4.3,Sevilla
R023,APT004,2025-06-15,2025-06-18,3,110.0,Airbnb,4.6,Sevilla
R024,APT008,2025-06-20,2025-06-22,1,70.0,Booking,4.2,Valencia
R025,APT005,2025-06-25,2025-06-28,2,85.0,Directo,4.8,Bilbao
```

Fichero 2 — `apartamentos.csv`

```
id_apartamento,tipo,habitaciones,metros_cuadrados,tiene_parking,propietario_id,activo
APT001,Estudio,1,35,false,P01,true
APT002,Piso,3,85,true,P02,true
APT003,Estudio,1,30,false,P01,true
APT004,Piso,2,65,true,P03,true
APT005,Ático,2,70,true,P04,true
APT006,Piso,3,90,true,P02,true
APT007,Estudio,1,28,false,P05,true
APT008,Estudio,1,32,false,P03,false
APT009,Piso,2,55,false,P04,true
APT010,Ático,3,100,true,P05,true
```

Fichero 3 — `propietarios.json`

```
[
  {"propietario_id": "P01", "nombre": "Laura Sánchez", "ciuda
d": "Madrid", "antiguedad_anios": 5, "tipo_contrato": "Premiu
m", "especialidades": ["Estudio","Piso"]},
  {"propietario_id": "P02", "nombre": "Carlos Méndez", "ciuda
d": "Barcelona", "antiguedad_anios": 3, "tipo_contrato": "Est
ándar", "especialidades": ["Piso"]},
  {"propietario_id": "P03", "nombre": "Ana Ferrero", "ciuda
d": "Sevilla", "antiguedad_anios": 7, "tipo_contrato": "Premi
um", "especialidades": ["Piso","Ático"]},
  {"propietario_id": "P04", "nombre": "Javier Ruiz", "ciuda
d": "Bilbao", "antiguedad_anios": 2, "tipo_contrato": "Estánd
ar", "especialidades": ["Ático","Piso"]},
```

```
  {"propietario_id": "P05", "nombre": "Marta Gil", "ciudad":
"Valencia", "antiguedad_anios": 6, "tipo_contrato": "Premiu
m", "especialidades": ["Estudio","Ático"]}
]
```

⚙️ Celda 0 — Inicialización del entorno (obligatoria)

Ejecuta esta celda antes de cualquier otra.

```
// Celda 0 — Inicialización
import $ivy.`org.apache.spark::spark-core:4.1.1`
import $ivy.`org.apache.spark::spark-sql:4.1.1`

import org.apache.spark.sql.SparkSession
import org.apache.spark.sql.functions._
import org.apache.spark.sql.expressions.Window
import org.apache.spark.sql.types._

val spark = SparkSession.builder()
  .appName("UrbanRent_Analytics")
  .master("local[*]")
  .config("spark.sql.shuffle.partitions", "4")
  .config("spark.sql.crossJoin.enabled", "true")
  .getOrCreate()

import spark.implicits._
spark.sparkContext.setLogLevel("ERROR")

println(s"✅ UrbanRent Analytics iniciado — Spark ${spark.ver
sion} · Scala ${scala.util.Properties.versionString}")
```

Salida esperada:

```
✅ UrbanRent Analytics iniciado — Spark 4.1.1 · Scala version 2.13.18
```

🗂️ PARTE 1 — Carga y exploración inicial

Pregunta 1 — ¿Están bien formateados nuestros datos de reservas?

El responsable de datos quiere verificar que Spark ha inferido correctamente los tipos antes de avanzar con cualquier análisis.

Lo que debes hacer:

1. Carga `reservas.csv` con `header = true` e `inferSchema = true`. Llama al DataFrame `reservas`.
2. Muestra las primeras 8 filas con `show(8)`.
3. Imprime el schema completo con `printSchema()`.
4. Muestra estadísticas descriptivas de `precio_noche`, `num_huespedes` y `valoracion` con `describe()`.
5. Imprime el número total de reservas con `count()` y lista todas las columnas con `columns`.

Salida esperada (parcial):

```
=== Primeras 8 reservas ===
+-----------+---------------+-------------+------------+--------------+------------+-----
--+----------+--------+
|id_reserva |id_apartamento |fecha_entrada |fecha_salida|num_huespedes |precio_noche|cana
l  |valoracion|ciudad  |
+-----------+---------------+-------------+------------+--------------+------------+-----
--+----------+--------+
|R001       |APT001         |2025-01-10   |2025-01-13  |2             |95.0        |Airbn
b |4.8       |Madrid  |
...

Total de reservas: 25
Columnas: id_reserva | id_apartamento | fecha_entrada | fecha_salida | num_huespedes | pr
ecio_noche | canal | valoracion | ciudad
```

Pregunta 2 — ¿Es correcto el schema inferido o necesitamos ajustarlo?

El equipo técnico señala que `fecha_entrada` y `fecha_salida` llegan como `StringType`. Para los cálculos de duración de estancias necesitarán ser `DateType`.

Lo que debes hacer:

1. Define un `StructType` manual para `reservas` con al menos estos ajustes:
   - `fecha_entrada` y `fecha_salida` como `DateType` (usa `to_date` o define el tipo en el schema).
   - `precio_noche` como `DoubleType`.
   - `valoracion` como `DoubleType`.
   - `num_huespedes` como `IntegerType`.
2. Recarga `reservas.csv` usando `.schema(...)` en lugar de `inferSchema`.
3. Verifica con `printSchema()` que los tipos son ahora los correctos.

Pista: Para leer fechas con schema manual en CSV usa `.option("dateFormat", "yyyy-MM-dd")`.

Pregunta 3 — ¿Cuántos apartamentos tenemos y cuáles son sus características?

Lo que debes hacer:

1. Carga `apartamentos.csv` con `inferSchema = true`. Llama al DataFrame `apartamentos`.
2. Carga `propietarios.json` con `multiline = true`. Llama al DataFrame `propietarios`.
3. Para cada uno: muestra el schema, las primeras filas y el `count()`.
4. ¿Cuántos apartamentos están marcados como `activo = true`?

🔧 PARTE 2 — Operaciones básicas sobre DataFrames

📌 Sesión cubierta: Clase 17 Sesión 1

Pregunta 4 — ¿Qué reservas corresponden a estancias de más de 3 noches?

La dirección quiere un informe de reservas largas para identificar clientes de alto valor.

Lo que debes hacer:

1. Sobre el DataFrame `reservas`, añade una columna `num_noches` calculada como `datediff($"fecha_salida", $"fecha_entrada")`.
2. Añade también una columna `ingreso_reserva` = `num_noches * precio_noche`.
3. Filtra para quedarte solo con reservas donde `num_noches > 3`.
4. Selecciona únicamente: `id_reserva`, `ciudad`, `canal`, `num_noches`, `ingreso_reserva`.
5. Ordena de mayor a menor `ingreso_reserva`.

Llama al resultado `reservasLargas` y muéstralo.

Pregunta 5 — ¿Qué canales de venta tenemos activos y cuál es el precio medio por canal?

El equipo de marketing quiere un catálogo limpio de canales sin repetidos.

Lo que debes hacer:

1. Sobre `reservas`, elimina duplicados por la columna `canal`.
2. Selecciona únicamente las columnas `canal` y `precio_noche`.
3. Ordena de mayor a menor `precio_noche`.

Pista: Usa `dropDuplicates("canal")`.

Pregunta 6 — ¿Cuánto ingresaría cada reserva si aplicáramos un suplemento del 10% para reservas con más de 2 huéspedes?

Lo que debes hacer:

1. Añade una columna `precio_con_suplemento` usando `when` / `otherwise`:
   - Si `num_huespedes > 2`: `precio_noche * 1.10`
   - En otro caso: `precio_noche`
2. Añade `ingreso_estimado` = `num_noches * precio_con_suplemento` (asegúrate de que `num_noches` existe en el DataFrame).

3. Selecciona `id_reserva`, `ciudad`, `num_huespedes`, `precio_noche`, `precio_con_suplemento`, `ingreso_estimado`.
4. Ordena por `ingreso_estimado` descendente.

Pregunta 7 — ¿Existen ciudades con reservas en el dataset de apartamentos que no aparecen en el de reservas?

Lo que debes hacer:

1. Renombra la columna `ciudad` del DataFrame de propietarios a `ciudad_propietario` para evitar ambigüedades.
2. Muestra las ciudades únicas en `reservas` con `select("ciudad").distinct()`.
3. Muestra las ciudades únicas en `propietarios` (columna `ciudad`) con `select("ciudad").distinct()`.

📊 PARTE 3 — Agregaciones y métricas de negocio

📌 Sesión cubierta: Clase 17 Sesión 2

Pregunta 8 — ¿Qué ciudad genera más ingresos para la empresa?

Lo que debes hacer:

1. Parte del DataFrame que ya tiene la columna `ingreso_reserva` (Pregunta 4, o recalcúlala si es necesario).
2. Agrupa por `ciudad` y calcula:
   - `num_reservas` = número de reservas
   - `ingreso_total` = suma de `ingreso_reserva`
   - `ticket_medio` = promedio de `ingreso_reserva`
   - `valoracion_media` = promedio de `valoracion`
3. Ordena por `ingreso_total` descendente.

Pregunta 9 — ¿Cuál es el canal de venta más rentable por ciudad?

La dirección quiere saber si Airbnb, Booking o el canal Directo domina en cada ciudad.

Lo que debes hacer:

1. Agrupa por `ciudad` y `canal`.
2. Calcula `ingreso_total` = suma de `ingreso_reserva` y `num_reservas` = count.
3. Ordena por `ciudad` y `ingreso_total` descendente.

Pregunta 10 — ¿Qué apartamento tiene la mejor posición de ingreso dentro de su ciudad?

Lo que debes hacer:

1. Calcula primero `ingreso_reserva` si no existe todavía.
2. Agrupa por `id_apartamento` y `ciudad`, calcula `ingreso_total` = suma de `ingreso_reserva`.
3. Define una ventana `ventanaCiudad` con `partitionBy("ciudad")` y `orderBy($"ingreso_total".desc)`.
4. Añade la columna `ranking_ciudad` usando `rank()` sobre esa ventana.
5. Filtra para ver solo el apartamento en posición 1 de cada ciudad.
6. Ordena por `ingreso_total` descendente.

Pregunta 11 — ¿Cuánto se ha facturado en total por ciudad y canal, con subtotales?

El CFO necesita un informe con subtotales por ciudad y también el gran total global para cuadrar con contabilidad.

Lo que debes hacer:

1. Usa `rollup("ciudad", "canal")` sobre el DataFrame con `ingreso_reserva`.
2. Agrega `sum("ingreso_reserva").as("total_facturado")`.
3. Ordena por `ciudad`, `canal`.
4. Muestra el resultado completo.

Reflexión: ¿Qué representan las filas donde `canal` es `null`? ¿Y la fila donde ambas columnas son `null`?

Pregunta 12 — ¿Podemos ver la facturación por ciudad y canal como una tabla cruzada?

El equipo de operaciones prefiere ver los datos en formato tabla dinámica para comparar canales entre ciudades de un vistazo.

Lo que debes hacer:

1. Usa `groupBy("ciudad").pivot("canal").agg(sum("ingreso_reserva"))`.
2. Ordena por `ciudad`.
3. Muestra el resultado.

Reflexión: ¿Qué aparece como `null` en la tabla pivot y qué significa?

🔗 PARTE 4 — Cruce de fuentes de datos (Joins)

📌 Sesión cubierta: Clase 18 Sesión 1

> 🔧 **Preparación:** Antes de esta sección, asegúrate de que los tres DataFrames base están cargados (`reservas`, `apartamentos`, `propietarios`) y de que existe la columna `ingreso_reserva` en `reservas`. Si `apartamentos` tiene una columna `ciudad`, renómbrala a `ciudad_apt` para evitar conflictos.

Pregunta 13 — ¿Cuántas noches y a qué precio se ha alquilado cada apartamento?

El equipo de producto quiere enriquecer el historial de reservas con información del apartamento (tipo, habitaciones, metros).

Lo que debes hacer:

1. Renombra la columna `activo` de `apartamentos` a `apt_activo` para evitar ambigüedades.
2. Haz un **INNER JOIN** entre `reservas` y `apartamentos` usando `id_apartamento` como clave.

3. Selecciona: `id_reserva`, `ciudad` (de reservas), `tipo`, `habitaciones`, `metros_cuadrados`, `precio_noche`, `ingreso_reserva`.
4. Ordena por `ingreso_reserva` descendente.
5. ¿Cuántas filas tiene el resultado? ¿Se pierde alguna reserva?

Pregunta 14 — ¿Hay apartamentos que nunca han tenido ninguna reserva?

El equipo de negocio necesita identificar activos improductivos para renegociar contratos con propietarios.

Lo que debes hacer:

1. Haz un **LEFT JOIN** entre `apartamentos` y `reservas` usando `id_apartamento` como clave.
2. Filtra las filas donde `id_reserva` sea `null` (estos son los apartamentos sin reservas).
3. Selecciona `id_apartamento`, `tipo`, `habitaciones`, `propietario_id`.
4. Muestra el resultado.

Reflexión: ¿Qué tipo de join usarías si en cambio quisieras ver todas las reservas aunque el apartamento ya no exista en el catálogo?

Pregunta 15 — ¿Qué propietarios tienen al menos un apartamento con reservas activas?

El departamento legal necesita contactar a los propietarios activos para renovar contratos.

Lo que debes hacer:

1. Haz un **LEFT SEMI JOIN** entre `propietarios` y `apartamentos` usando `propietario_id` como clave.
2. Muestra solo las columnas de `propietarios`: `propietario_id`, `nombre`, `tipo_contrato`.

Pregunta 16 — ¿Qué propietarios no tienen ningún apartamento en el catálogo actual?

Lo que debes hacer:

1. Haz un **LEFT ANTI JOIN** entre `propietarios` y `apartamentos` usando `propietario_id` como clave.
2. Muestra `propietario_id`, `nombre`, `ciudad`.

Pregunta 17 — ¿Cuál es el informe completo de reservas con datos de apartamento y propietario?

El equipo directivo quiere un informe consolidado que cruce las tres fuentes.

Lo que debes hacer:

1. Empieza por el INNER JOIN de `reservas` con `apartamentos` (Pregunta 13, renombrando columnas ambiguas antes).
2. Encadena un segundo JOIN (tipo `inner`) con `propietarios` usando `propietario_id`.
3. Selecciona: `id_reserva`, `ciudad` (reservas), `tipo`, `habitaciones`, `nombre` (propietario), `tipo_contrato`, `ingreso_reserva`.
4. Ordena por `ingreso_reserva` descendente.

Pregunta 18 — ¿Tiene sentido aplicar broadcast en alguno de estos joins?

El equipo técnico quiere documentar las decisiones de optimización.

Lo que debes hacer:

1. Repite el JOIN de `reservas` con `apartamentos` **con** `broadcast(apartamentos)`.
2. Ejecuta `.explain()` sobre ambas versiones (sin broadcast y con broadcast).
3. Responde en una celda Markdown:
   - ¿Qué tipo de join aparece en el plan sin broadcast?
   - ¿Qué tipo de join aparece en el plan con broadcast?

- ¿Por qué tiene sentido hacer broadcast sobre `apartamentos` y no sobre `reservas`?

🔠 PARTE 5 — Funciones integradas y UDFs

📌 Sesión cubierta: Clase 19 Sesión 1

Pregunta 19 — ¿Cómo podemos presentar las fechas de entrada en formato europeo?

El equipo de atención al cliente necesita las fechas en formato `dd/MM/yyyy` para los informes que envían a los propietarios.

Lo que debes hacer:

1. Sobre `reservas`, añade la columna `fecha_entrada_es` usando `date_format($"fecha_entrada", "dd/MM/yyyy")`.
2. Añade también `mes_entrada` = `month($"fecha_entrada")` y `anio_entrada` = `year($"fecha_entrada")`.
3. Selecciona `id_reserva`, `ciudad`, `fecha_entrada`, `fecha_entrada_es`, `mes_entrada`, `anio_entrada`.
4. Muestra las primeras 6 filas.

Pregunta 20 — ¿Cuántos días de antelación se hacen las reservas respecto a una fecha de referencia?

Supon que hoy es `2025-01-01` (fecha de referencia fija para que el resultado sea reproducible).

Lo que debes hacer:

1. Define una columna `fecha_ref` = `to_date(lit("2025-01-01"), "yyyy-MM-dd")`.
2. Calcula `dias_antelacion` = `datediff($"fecha_entrada", $"fecha_ref")`.
3. Selecciona `id_reserva`, `ciudad`, `fecha_entrada`, `dias_antelacion`.
4. Ordena por `dias_antelacion` ascendente (las reservas más próximas primero).

Pregunta 21 — ¿Están bien formateados los IDs de reserva? ¿Podemos estandarizarlos?

El sistema legado generó algunos IDs con espacios o en minúsculas. Aplica limpieza estándar.

Lo que debes hacer:

1. Simula datos sucios añadiendo una columna `id_reserva_sucio` con: `concat($"id_reserva", lit(" "))` (añade un espacio al final).
2. Limpia esa columna con `trim()` y ponla en mayúsculas con `upper()`. Guarda el resultado como `id_reserva_limpio`.
3. Añade también una columna `ciudad_normalizada` = `initcap($"ciudad")`.
4. Selecciona las tres versiones para comparar: `id_reserva_sucio`, `id_reserva_limpio`, `ciudad`, `ciudad_normalizada`.
5. Muestra las primeras 5 filas.

Pregunta 22 — ¿Qué especialidades tienen registradas los propietarios?

El JSON de propietarios incluye un array `especialidades` por propietario. El equipo quiere analizar estas especialidades a nivel de fila individual.

Lo que debes hacer:

1. Sobre `propietarios`, usa `explode($"especialidades")` para expandir el array en una fila por especialidad. Llama a la nueva columna `especialidad`.
2. Selecciona `propietario_id`, `nombre`, `especialidad`.
3. Muestra el resultado.
4. ¿Cuántas filas tiene el resultado después del explode?
5. Cuenta cuántos propietarios tienen cada `especialidad` usando `groupBy("especialidad").count()`. Ordena de mayor a menor.

Pregunta 23 — ¿Cómo clasificamos la calidad de cada reserva según su valoración?

El equipo de calidad necesita una categorización automática para filtrar reservas en los informes.

Lo que debes hacer:

1. Crea una UDF llamada `clasificarCalidadUDF` que reciba `valoracion: java.lang.Double` y devuelva:

Condición	Categoría
`null`	`"Sin valoración"`
`>= 4.8`	`"Excelente"`
`>= 4.5`	`"Muy buena"`
`>= 4.0`	`"Buena"`
cualquier otro	`"Mejorable"`

2. Aplica la UDF sobre la columna `valoracion` del DataFrame `reservas` y añade la columna `calidad_reserva`.
3. Registra la UDF en Spark SQL con `spark.udf.register("clasificar_calidad", clasificarCalidadUDF)`.
4. Selecciona `id_reserva`, `ciudad`, `valoracion`, `calidad_reserva` y muestra el resultado completo.
5. Agrupa por `calidad_reserva` y cuenta cuántas reservas caen en cada categoría. Ordena de mayor a menor.

Pregunta 24 — Informe final consolidado con Spark SQL

Registra el DataFrame de reservas enriquecidas (con `ingreso_reserva`, `calidad_reserva`, `mes_entrada`) como vista temporal y ejecuta la siguiente consulta SQL para producir el informe de cierre.

Lo que debes hacer:

1. Sobre el DataFrame `reservas` añade las columnas `ingreso_reserva`, `calidad_reserva` y `mes_entrada` (si no existen ya).
2. Regístralo como vista temporal: `reservasEnriquecidas.createOrReplaceTempView("reservas_enriquecidas")`.

3. Ejecuta esta consulta con `spark.sql(...)`:

```sql
SELECT
  ciudad,
  canal,
  COUNT(*) AS num_reservas,
  ROUND(SUM(ingreso_reserva), 2) AS ingreso_total,
  ROUND(AVG(valoracion), 2) AS valoracion_media,
  clasificar_calidad(AVG(valoracion)) AS calidad_media
FROM reservas_enriquecidas
GROUP BY ciudad, canal
ORDER BY ingreso_total DESC
```

4. Muestra el resultado.

✅ Verificación final

Ejecuta esta celda al finalizar para validar los resultados clave del caso de estudio.

```
println("=" * 60)
println("VERIFICACIÓN FINAL — Caso de Estudio UrbanRent Analy
tics")
println("=" * 60)

// Ajusta los nombres de variable a los que hayas usado en tu
solución
val comprobaciones = Seq(
  ("P1  — Total reservas cargadas (25)",                  res
ervas.count() == 25),
  ("P3  — Apartamentos activos (9)",                      apa
rtamentos.filter($"activo" === true).count() == 9),
  ("P4  — Reservas largas > 3 noches",                    res
ervasLargas.count() > 0),
  ("P8  — Ciudades en el ranking de ingresos (5)",        res
```

```
ervas.select("ciudad").distinct().count() == 5),
  ("P13 — INNER JOIN reservas × apartamentos (25 filas)", res
ervas.join(apartamentos, "id_apartamento").count() == 25),
  ("P22 — Explode especialidades propietarios (> 5 filas)", p
ropietarios.select(explode($"especialidades")).count() >= 5)
)

comprobaciones.foreach { case (desc, ok) =>
  println(s"${if (ok) "✅ CORRECTO" else "❌ REVISAR"} — $des
c")
}
```